



UNIVERSITY OF CAPE TOWN

CSC200SS: Computer Science

Author: George Goodey

September 28, 2026

Contents

1	Parallel and Concurrent Programming	2
2	Theory of Computation	2
2.1	Regular Languages	2
2.1.1	Finite automata	2
2.1.2	Strings and languages	3
2.1.3	Regular expressions	3
2.1.4	Nondeterministic finite automata	4
2.1.5	Converting NFAs to DFAs	5
2.1.6	Closure properties for regular languages	5
2.1.7	Converting a regular expression to NFA	6
2.1.8	Generalized NFA	8
2.1.9	Converting a GNFA to a regular expression	9
2.1.10	Non-regular languages	9
2.1.11	Proving non-regularity	10
2.2	Context-free languages	11
2.2.1	Context-free grammars	11

1 Parallel and Concurrent Programming

2 Theory of Computation

2.1 Regular Languages

2.1.1 Finite automata

Definition: Finite automata

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

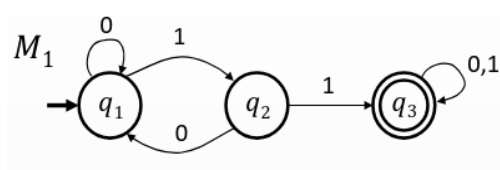
- Q is a finite set of all possible states.
- Σ is a finite set of all alphabet symbols.
- δ is a transition function $\delta : Q \times \Sigma \rightarrow Q$. In other words,

$$\delta(q, a) = r \text{ means } q \xrightarrow{a} r$$

- q_0 is the start state.
- F is the set of accept states.

Example:

Let's consider the following finite automaton.



Here, $M_1 = (Q, \Sigma, \delta, q_1, F)$ is a finite automaton. For sake of example, here are the exact details of M_1 .

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- $F = \{q_3\}$

It accepts a finite string as input. The output is either *ACCEPT* or *REJECT*. The computation goes as follows:

1. Begin at start state.
2. Read input symbols.
3. Follow corresponding transitions.
4. *ACCEPT* if we reach the accept state, otherwise *REJECT*.

For example: $01101 \rightarrow \text{ACCEPT}$, while $00101 \rightarrow \text{REJECT}$. Here, M_1 accepts all string in A where $A = \{w | w \text{ contains substring } 11\}$. We say that A is the language of $M_1 \iff M_1$ recognizes $A \iff A = L(M_1)$.

2.1.2 Strings and languages

A **string** is a finite sequence of symbols $w_1w_2\cdots w_n \in \Sigma$. A **language** is a set of string (finite or infinite). The **empty string** ε is the string of length 0. The **empty language** $\emptyset$ is the set with no strings.

Definition: Accepting strings

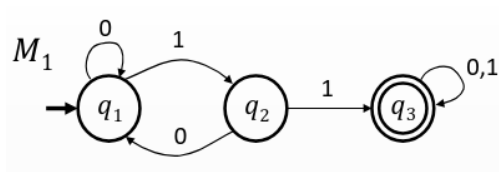
A finite automaton M **accepts string** $w = w_1w_2\cdots w_n$ for each $w_i \in \Sigma$ if there exists a sequence of states $r_0, r_1, \dots, r_n \in Q$ where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \in F$

Definition: Regular language

We say a language is **regular** if some finite automaton recognizes it.

Example 1:



$L = (M_1) = \{w|w \text{ contains substring } 11\} = A$. Therefore, A is regular.

Example 2:

Let $B = \{w|w \text{ has an even number of 1s}\}$. In this case, B is regular. (not shown)

Example 3:

Let $C = \{w|w \text{ has equal number of 0s and 1s}\}$. In this case, C is not regular. (provable)

2.1.3 Regular expressions

Definition: Regular operations

Let A, B be languages.

- **Union:** $A \cup B = \{w|w \in A \text{ or } w \in B\}$.
- **Concatenation:** $A \circ B = \{xy|x \in A \text{ and } y \in B\} = AB$.
- **Star:** $A^* = \{x_1 \dots x_k|x_i \in A \text{ for } k \geq 0\}$. (Note: $\varepsilon \in A^*$)

Regular expressions are built from $\Sigma, \emptyset, \varepsilon$ by using regular operations $\cup, \circ, \star$.

Example 1:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ .
- Σ^*1 gives all strings that end with 1.
- $\Sigma^*11\Sigma^*$ gives all strings that contain 11 = $L(M_1)$.

Example 2:

Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$.
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$.
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, ...}\}$

2.1.4 Nondeterministic finite automata**Definition: Nondeterministic finite automata**

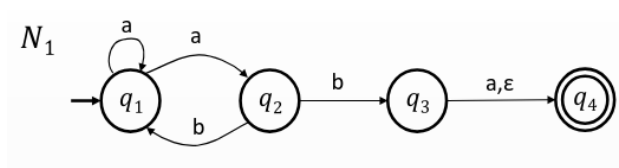
A **nondeterministic finite automata (NFA)** N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where everything is the exact same as a DFA except δ . Here, $\delta : Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q) = \{R \mid R \subseteq Q\}$.

New features of nondeterminism:

- Multiple paths possible (0, 1, or many at each step)
- ε -transition is a "free" move without reading input.
- Accept input if **some** path leads to the accept state.

Nondeterminism doesn't correspond to a physical machine we can build. However, it is useful mathematically. Here are some ways to think about it:

- **Computationally:** Fork new parallel thread and accept if any thread leads to an accept state.
- **Mathematically:** Tree with branches. Accept if any branch leads to an accept state.
- **Magical:** Guess at each nondeterministic step which way to go. The machine always makes the right guess that leads to the accept state, if possible.

Example:

Here, we have $\delta(q_1, a) = \{q_1, q_2\}$, but $\delta(q_1, b) = \emptyset$.

2.1.5 Converting NFAs to DFAs

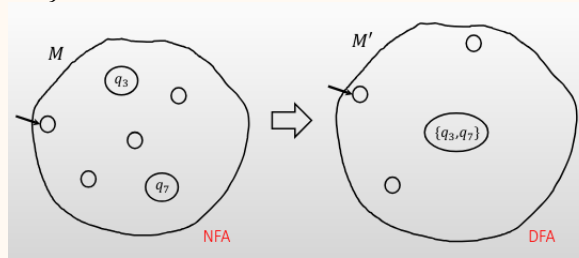
Theorem

If an NFA recognizes A , then A is regular.

Proof:

Let NFA $N = (Q, \Sigma, \delta, q_0, F)$ recognize A . We need to construct a DFA $M = (Q', \Sigma, \delta', q'_0, F')$, ignoring the ε -transitions. Here M is defined as:

- $Q' = \mathcal{P}(Q)$.
- $\delta'(R \in Q', a) = q \in \delta(r, a)$ for some $r \in R$.
- $q'_0 = \{q_0\}$.
- $F' = \{R \in Q' \mid R \cap F\}$.



2.1.6 Closure properties for regular languages

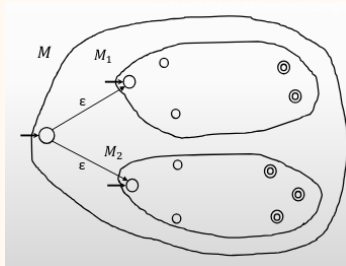
Theorem: Closure under $\cup$.

If A_1 and A_2 are regular languages, then so is $A_1 \cup A_2$.

Proof:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 . We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where

- $Q = Q_1 \times Q_2 = \{(q_1, q_2) \mid q_1 \in Q_1 \wedge q_2 \in Q_2\}$.
- $q_0 = (q_1, q_2)$.
- $\delta((q, r), a) = (\delta_1(q, a), \delta_2(q, a))$.
- $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.

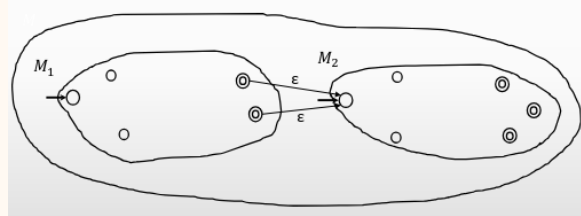


Theorem: Closure under $\circ$.

If A_1 and A_2 are regular languages, then so is A_1A_2 .

Proof:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 . We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where



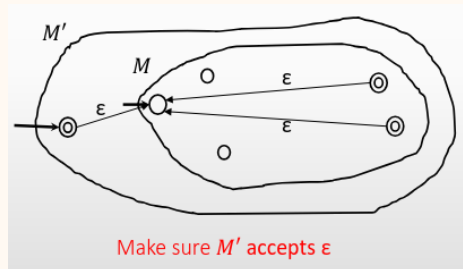
M should accept input w if $w = xy$ where M_1 accepts x and M_2 accepts y . The nondeterministic M has the option to jump to M_2 when M_1 accepts.

Theorem: Closure under $\star$

If A is a regular language, then so is $A^\star$.

Proof:

Given a DFA M recognizing A , construct an NFA M' recognizing $A^\star$.



M' should accept input w if $w = x_1x_2 \dots x_k$ where $k \geq 0$ and M accepts each x_i .

2.1.7 Converting a regular expression to NFA

Theorem

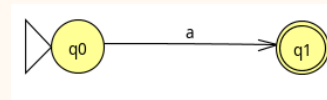
If R is a regular expression and $A = L(R)$, then A is regular.

Proof:

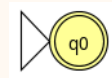
Convert R to the equivalent NFA M .

If R is atomic, an equivalent M is:

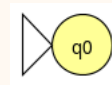
- $R = a$ for $a \in \Sigma$



- $R = \varepsilon$



- $R = \emptyset$



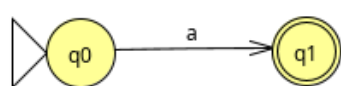
If R is composite:

- $R = R_1 \cup R_2$
- $R = R_1 \circ R_2$
- $R = R_1^*$

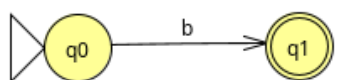
Example:

Convert $(a \cup ab)^*$ to an equivalent NFA.

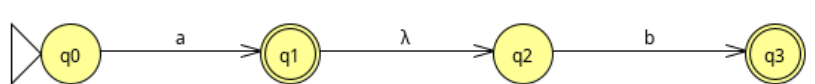
- a :



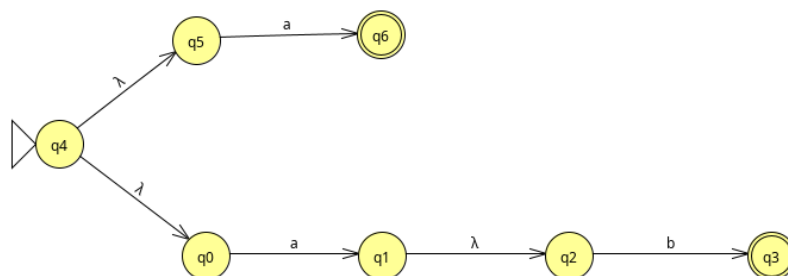
- b :



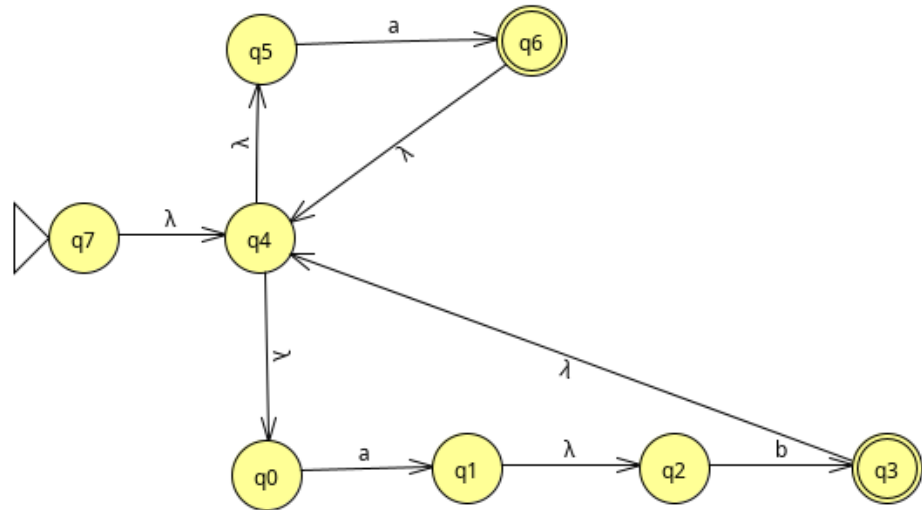
- ab :



- $a \cup ab$:



- $(a \cup ab)^*$:



Reminder

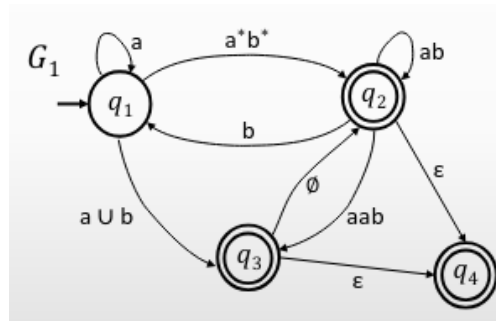
A λ transition is equivalent to an ε transition.

2.1.8 Generalized NFA

Definition: Generalized NFA

A **generalized nondeterministic finite automaton (GNFA)** is similar to an NFA, but allows regular expressions as transition labels.

Example:



For convenience, we will assume:

- One accept state, separate from the start state.
- One arrow from each state to each state, except
 - only exiting the start state
 - only entering the accept state

We can easily modify a GNFA to have this special form.

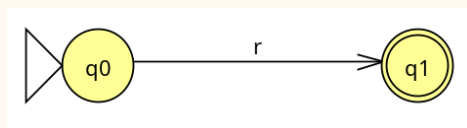
2.1.9 Converting a GNFA to a regular expression

Lemma

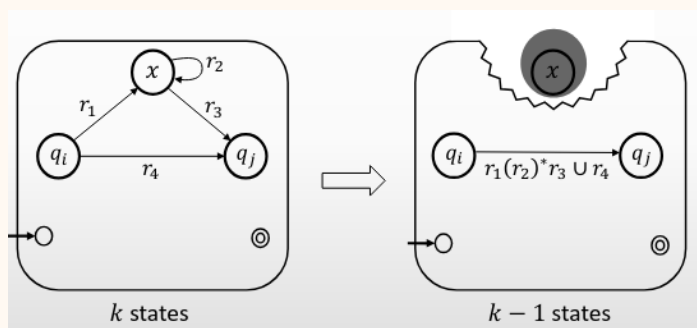
Every GNFA G has an equivalent regular expressions R .

Proof:

We will make use of an inductive argument on the number of states k of G . For the base step ($k = 2$):



Assume the Lemma holds true for $k - 1$ states. Now we want to prove it holds for k states. The central idea is to convert a k -state GNFA to an equivalent $(k - 1)$ -state GNFA.



1. Pick any state x except the start and accept states.
2. Remove x .
3. Repair the damage by recovering all paths that went through x .
4. Make the indicated change for each pair of states q_i, q_j .

Thus DFAs and regular expressions are equivalent. ■

2.1.10 Non-regular languages

How do we show a language is not regular?

- Remember, to show a language is *regular*, we give an equivalent DFA.
- To show a language is *not regular*, we must give a proof.
- It is not enough to say you couldn't find a DFA for it, therefore it isn't regular!

Example 1:

Let $B = \{w | w \text{ has equal numbers of 0s and 1s}\}$. One's intuition says that B is not regular since DFAs cannot count unboundedly.

Example 2:

Let $C = \{w \mid w \text{ has equal numbers of } 01 \text{ and } 10 \text{ substrings}\}$. As examples, $0101 \notin C$ and $0110 \in C$. Again, one's intuition says that C is not regular since DFA cannot count unboundedly. However, C is regular!

2.1.11 Proving non-regularity**Pumping Lemma**

For every regular language A , there exists a number p , called the "pumping length", such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

$$1. \ xy^iz \in A \text{ for all } i \geq 0 \quad (y^i = \underbrace{yy \cdots y}_i).$$

$$2. \ y \neq \varepsilon.$$

$$3. \ |xy| \leq p.$$

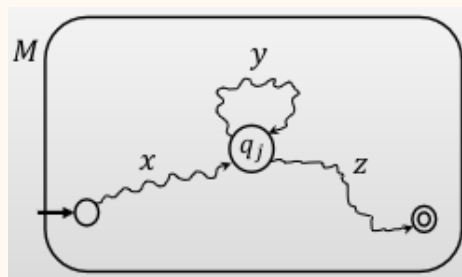
Informally, if A is regular, then every long string in A can be "pumped" and the result stays in A .

Proof:

Let DFA M recognize A . Let p be the number of states in M . Pick $s \in A$ where $|s| \geq p$.

$$s = \begin{array}{c} x \quad y \quad z \\ \hline q_j \quad q_j \end{array}$$

M will repeat a state q_j when reading s because s is so long. The path that M follows when reading s .

**Example 1:**

Let $D = \{0^k 1^k \mid k \geq 0\}$. Show that D is not regular.

Assume, for the sake of contradiction, that D is regular.

Since D is regular, the pumping lemma must hold. Let $s = 0^p 1^p \in D$.

The pumping lemma says that we can divide $s = xyz$.

But $xyyz$ has excess 0s, and thus $xyyz \notin D$ contradicting the pumping lemma.

Therefore, our assumption is false and thus D is not regular.

Example 2:

Let $F = \{ww \mid w \in \Sigma^*\}$. Say $\Sigma^* = \{0, 1\}^*$. Show that F is not regular.

Assume, for the sake of contradiction, that F is regular.

Since F is regular, the pumping lemma must hold.

We need to choose $s \in F$. Let $s = 0^p 10^p 1 \in F$.

We can show that $s = xyz$ cannot be pumped satisfying the 3 conditions.

Thus, $xyyz \notin F$, which is a contradiction. Therefore, F is not regular.

Example 3:

Let $B = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$. Show that B is not regular.

Assume, for the sake of contradiction, that B is regular.

We know 0^*1^* is regular, and so $B \cap 0^*1^*$ is regular (closure under intersection).

But $D = B \cap 0^*1^*$ and we already showed D is not regular (from example 1).

Therefore, B is not regular.

2.2 Context-free languages**2.2.1 Context-free grammars****Definition: Context-free grammar**

A **Context Free Grammar (CFG)** G is a 4-tuple (V, Σ, R, S) where

- V is the finite set of variables.
- Σ is the finite set of terminal symbols.
- R is the finite set of rules (rule form: $V \rightarrow (V \cup \Sigma)^*$).
- S the starting variable.

For $u, v \in (V \cup \Sigma)^*$ write

1. $u \Rightarrow v$ if you can go from u to v with one substitution step in G .
2. $u \xRightarrow{*} v$ if you can go from u to v with some number of substitution steps in G .
 $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v$ is called a *derivation* of v from u . Furthermore, if $u = S$ then it is a *derivation* of v .

$L(G) = \{w \mid w \in \Sigma^* \text{ and } S \xRightarrow{*} w\}$.

Definition: Context-free language

A is a **Context Free Language (CFL)** if $A = L(G)$ for some CFG G .